



Ahsanullah University of Science & Technology

Department of Computer Science & Engineering

Course No : CSE 3208
Course Title : Introduction to Artificial Intelligence Lab
Assignment No : 04

Date of Submission : 16.02.2026

Submitted To : Dr. S.M.A. Al-Mamun & Ms. Nawrin Tabassum

Submitted By-

Group : A1
Name : Nazim Raian Khan
Id : 20220204026
Section : A

1. With the help of the supplementary materials and demonstrated codes implement the variants of hill-climbing and genetic algorithms discussed above in Python.

ANS:

```
import random
```

```
# Utility Functions
```

```
def generate_random_state():  
    return [random.randint(1, 8) for _ in range(8)]
```

```
def calculate_fitness(state):  
    conflicts = 0  
    n = 8  
    for i in range(n):  
        for j in range(i + 1, n):  
            # Same row  
            if state[i] == state[j]:  
                conflicts += 1  
            # Same diagonal  
            if abs(state[i] - state[j]) == abs(i - j):  
                conflicts += 1  
    return 28 - conflicts
```

```
def generate_successors(state):  
    successors = []  
    for col in range(8):  
        for row in range(1, 9):  
            if row != state[col]:  
                new_state = state.copy()  
                new_state[col] = row  
                successors.append(new_state)  
    return successors
```

Basic Hill Climbing

```
def hill_climbing(initial_state, threshold=28):
    current = initial_state
    iteration = 0

    while True:
        current_fitness = calculate_fitness(current)
        if current_fitness >= threshold:
            print("\nFound Solution:", current, "Fitness:", current_fitness)
            return current

        successors = generate_successors(current)
        best_successor = max(successors, key=calculate_fitness)
        best_fitness = calculate_fitness(best_successor)

        if best_fitness <= current_fitness:
            print("\nStuck at Local Maximum:", current, "Fitness:", current_fitness)
            return None

        print("Iteration max:", best_fitness)
        current = best_successor
        iteration += 1
```

Random Restart Hill Climbing

```
def random_restart_hill_climbing(threshold=28, max_restarts=5):
    for restart in range(max_restarts):
        print("\nRestart:", restart + 1)
        state = generate_random_state()
        result = hill_climbing(state, threshold)
        if result:
            return result
    print("\nFailed after all restarts.")
```

```
return None
```

Stochastic Hill Climbing

```
def stochastic_hill_climbing(initial_state, threshold=28):
    current = initial_state

    while True:
        current_fitness = calculate_fitness(current)
        if current_fitness >= threshold:
            print("\nFound Solution:", current)
            return current

        successors = generate_successors(current)
        better_successors = [s for s in successors if calculate_fitness(s) >
current_fitness]

        if not better_successors:
            print("\nStuck at Local Maximum.")
            return None

        current = random.choice(better_successors)
        print("Moved to:", current, "Fitness:", calculate_fitness(current))
```

First-Choice Hill Climbing

```
def first_choice_hill_climbing(initial_state, threshold=28):
    current = initial_state

    while True:
        current_fitness = calculate_fitness(current)
        if current_fitness >= threshold:
            print("\nFound Solution:", current)
            return current
```

```
found_better = False
```

```
for _ in range(100): #trying 100 random successor
```

```
    col = random.randint(0, 7)
```

```
    row = random.randint(1, 8)
```

```
    if row != current[col]:
```

```
        new_state = current.copy()
```

```
        new_state[col] = row
```

```
        if calculate_fitness(new_state) > current_fitness:
```

```
            current = new_state
```

```
            found_better = True
```

```
            print("Moved to:", current, "Fitness:", calculate_fitness(current))
```

```
            break
```

```
if not found_better:
```

```
    print("\nStuck at Local Maximum.")
```

```
    return None
```

```
# Genetic Algorithm
```

```
def crossover(parent1, parent2):
```

```
    point = random.randint(1, 7)
```

```
    child1 = parent1[:point] + parent2[point:]
```

```
    child2 = parent2[:point] + parent1[point:]
```

```
    return child1, child2
```

```
def mutate(state, mutation_rate=0.1):
```

```
    if random.random() < mutation_rate:
```

```
        col = random.randint(0, 7)
```

```
        state[col] = random.randint(1, 8)
```

```
    return state
```

```

def genetic_algorithm(pop_size=8, threshold=28, generations=1000):
    population = [generate_random_state() for _ in range(pop_size)]

    for gen in range(generations):
        population.sort(key=calculate_fitness, reverse=True)

        if calculate_fitness(population[0]) >= threshold:
            print("\nSolution Found in Generation", gen)
            print("State:", population[0], "Fitness:",
calculate_fitness(population[0]))
            return population[0]

        print("Generation", gen, "Best Fitness:", calculate_fitness(population[0]))

        next_generation = population[:2]

        while len(next_generation) < pop_size:
            parent1 = random.choice(population[:4])
            parent2 = random.choice(population[:4])
            child1, child2 = crossover(parent1, parent2)
            next_generation.append(mutate(child1))
            if len(next_generation) < pop_size:
                next_generation.append(mutate(child2))

        population = next_generation

    print("\nNo solution found.")
    return None

# MAIN

if __name__ == "__main__":

```

```
initial = [2,3,4,5,6,5,7,8]

print("\n--- Basic Hill Climbing ---")
hill_climbing(initial, 27)

print("\n--- Random Restart Hill Climbing ---")
random_restart_hill_climbing(28)

print("\n--- Stochastic Hill Climbing ---")
stochastic_hill_climbing(initial, 28)

print("\n--- First Choice Hill Climbing ---")
first_choice_hill_climbing(initial, 28)

print("\n--- Genetic Algorithm ---")
genetic_algorithm()
```